

GROUPE 1 · PROJET LOGISTIQUE

Blueprint Technique & Architectural

Assistant Logistique de Poche - Conception, Infrastructure et Synchronisation Offline-First

Champ	Détail
Projet	Assistant Logistique de Poche
Organisation	Groupe 1
Période	6 Mai – 06 Juillet 2026 (8 semaines)
Stack cible	NextJs · NestJS · PostgreSQL
Équipe	7 membres

SECTION 1

L'Impératif logistique du voyageur d'affaires

Le cœur métier du projet repose sur le calcul automatisé du **Landed Cost (Vrai Prix)**, soit la somme exacte qu'un article coûte une fois toutes les charges imputées.

Composante	Description	Support technique
Prix d'achat	Prix local de la marchandise au moment de l'acquisition	Table <code>Items</code> - <code>local_price</code> , <code>local_currency</code>
Frais de douanes	Taxes et droits liés au passage en douane	Table <code>Expenses</code> - <code>expense_type</code> = <code>douanes</code>
Transport & billets	Coûts logistiques d'acheminement des colis	Table <code>Expenses</code> - <code>expense_type</code> = <code>transport</code>

Défis couverts par l'architecture

- **Multiplicité des devises** : Conversion instantanée et automatisée en Francs CFA. Le taux de change est figé en base de données via la colonne `exchange_rate_at_purchase` de la table `Items`.
- **Répartition des frais** : Imputation algorithmique des coûts annexes sur le prix de revient unitaire. La table `Expenses` centralise les frais par voyage et par colis pour un calcul précis au prorata.
- **Environnements hostiles** : Architecture résiliente pour les entrepôts et marchés sans couverture réseau - traité par le mode Offline-First (voir Section 8).

SECTION 2

Frontières fonctionnelles et rôles des acteurs

Le diagramme de cas d'utilisation a été mis en conformité avec les standards UML. Les corrections suivantes ont été appliquées :

Corrections UML appliquées

- **Correction des relations Extends** : Les flèches d'extension partent des cas d'utilisation secondaires (*Pour moi* et *Pour clients*) pour pointer vers le cas d'utilisation de base.
- **Correction orthographique** : Nettoyage des libellés en "Pour clients" sur l'ensemble des modules.
- **Rattachement du cas "Historiques"** : Liaison directe par un trait d'association simple entre l'acteur **User** et le cas d'utilisation **Historiques**.
- **Simplification globale** : Suppression des dépendances `include` redondantes vers *Authentifier* pour centraliser la condition d'accès en amont de l'application.

Périmètres des acteurs

Acteur	Périmètre
User	Voyageur/Acheteur - Opérations locales et logistiques : gestion des produits, stocks, catégories, comptabilité et historiques.
Admin	Superviseur - Contrôle global de la plateforme.
Tronc Commun	Sécurité croisée obligatoire via le flux central d'authentification.

SECTION 3

Matrice d'allocation technologique

La méthodologie appliquée est **Function-to-Infrastructure** : chaque exigence métier dicte un composant logiciel précis, éliminant toute sur-ingénierie.

Besoin Fonctionnel (UML)	Stack Technologique	Justification
Authentifier les acteurs	NestJS + Passport JWT	Sécurité API - Guards NestJS bloquant l'accès si le JWT est absent ou altéré.
Gérer les produits (Offline)	NextJs + Dexie.js + Workbox	PWA & Cache - stockage local IndexedDB et service worker pour fonctionnement sans réseau.
Gérer les catégories & relations	Prisma ORM	Type-safety absolu - génération automatique du client TypeScript depuis le schéma.
Historiques & Intégrité financière	PostgreSQL	SGBD Relationnel - stockage persistant, intégrité transactionnelle et agrégations du Landed Cost.

Écosystème technologique conteneurise

L'ensemble de l'infrastructure est orchestré via **Docker**, garantissant l'isolation et la reproductibilité entre les environnements de développement et de production.

Couche Client / Présentation

NextJs bootstrappé avec **Vite**. Utilisation du plugin `vite-plugin-pwa` pour transformer l'application en Application Web Progressive (PWA) installable sur mobile. Interface utilisateur asynchrone et offline-ready.

Couche API / Business

Node.js avec le framework **NestJS** en TypeScript, garantissant une architecture modulaire et maintenable. Cette couche est responsable du traitement du Landed Cost et de la sécurisation des routes.

Couche Base de Données

PostgreSQL interfacé via **Prisma ORM**. Assure le stockage persistant et l'intégrité relationnelle. Déployé via un conteneur Docker en phase de développement.

Briques complémentaires

- **Dexie.js** (surcouche à IndexedDB) : stockage local des données sur le smartphone avant synchronisation, en mode hors-ligne.
- **Workbox BackgroundSyncPlugin** : gestion de la file d'attente des requêtes en mode offline et rejeu automatique au retour du réseau.

Bouclier d'authentification NestJS & Passport

Le système d'authentification est basé sur **JWT (JSON Web Tokens)** avec la librairie Passport. Le flux se déroule en cinq étapes :

Flux d'authentification

1. **Client NextJs** → **POST /login** : Envoi des identifiants (credentials) depuis l'interface utilisateur.
2. **NestJS AuthGuard('local')** : Validation des identifiants hashés via `LocalStrategy`.
3. **Service JWT** : Génération et signature du Token éphémère.
4. **Client NextJs** : Toute requête ultérieure inclut le header `Bearer <token>`.
5. **NestJS AuthGuard('jwt')** : Protection des routes critiques (ex. : création de colis).

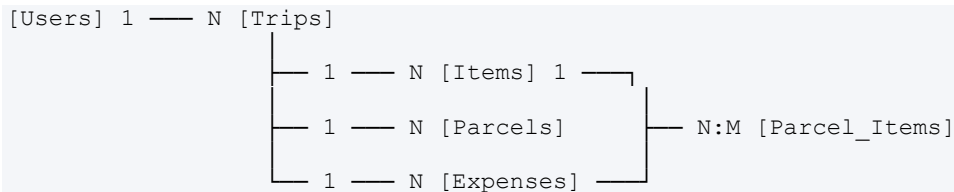
Sécurité par Décorateurs L'accès aux endpoints métier est strictement bloqué par les Guards NestJS si le JWT est absent ou altéré. Le système d'authentification constitue un livrable du Sprint 1.

SECTION 6

Modélisation relationnelle avec prisma

Le modèle de données a été restructuré afin de garantir l'exactitude comptable et de supporter le fractionnement des articles dans plusieurs colis.

Schéma des entités et relations



Description des tables

#	Table	Colonnes clés	Rôle
1	Users	id, email, password_hash, default_currency	Informations d'authentification et préférences système.
2	Trips	id, name, start_date, end_date, user_id	Conteneur temporel et logique des opérations.
3	Items	id, name, local_price, local_currency, exchange_rate_at_purchase, trip_id	Liste des marchandises. Fige le taux de change CFA à la seconde précise de l'achat.
4	Parcels	id, tracking_number, status, origin_place, destination_place, trip_id	Unités logistiques d'expédition.
5	Parcel_Items	parcel_id, item_id, quantity_assigned	Table de jointure M:N entre articles et colis. Permet de diviser un lot dans différents colis.
6	Expenses	id, amount_cfa, expense_type, trip_id, parcel_id	Centralise les coûts de transport, douanes et logistique. Calcul au prorata du Vrai Prix.

Avantages de Prisma ORM

- Le schéma génère automatiquement un client TypeScript typé.
- Garantit un typage fort de bout en bout entre la base PostgreSQL et l'API NestJS.
- Élimine les erreurs de casting lors des requêtes d'agrégation du Landed Cost.

Résolution des relations complexes (Many-to-Many)

Type	Cas d'usage	Mécanisme
Relation M-N Implicite	Lier simplement Produits et Catégories.	Prisma gère la table de jointure en arrière-plan (ex. : <code>_CategorieToProduit</code>).
Relation M-N Explicite	Expéditions complexes avec métadonnées métier (quantité, date d'assignation).	Table de jointure dédiée Parcel_Items avec colonne <code>quantity_assigned</code> .

Choix Architectural L'utilisation de relations Many-to-Many explicites est privilégiée pour les expéditions complexes afin de tracer l'historique complet d'assignation des colis.

Le Défi critique du Offline-First

Contexte métier

La saisie des marchandises est requise dans des hangars ou à l'étranger sans couverture de données mobiles (roaming data). L'application doit rester pleinement opérationnelle dans ces environnements hostiles.

Mandat technique

L'application NextJs doit se comporter comme une application native. La création de colis et la saisie de produits sont garanties sans aucune connexion réseau, grâce à la combinaison de **Dexie.js** (IndexedDB) et du **Service Worker Workbox**.

Solution implémentée

- **Dexie.js** (IndexedDB) : stockage local sur le smartphone.
- **Workbox BackgroundSyncPlugin** : intercepte et met en file d'attente les requêtes POST/PUT.
- Rejeu automatique des opérations lors du retour de la connexion réseau.

Architecture de synchronisation asynchrone

Le cycle de synchronisation se déroule en cinq étapes, formant une boucle fermée entre l'interface utilisateur et la base de données persistante.

6. **Saisie NextJs (UI)** : Le hook `useLiveQuery` de Dexie assure la réactivité immédiate de l'interface.
7. **Stockage local** : Dexie.js (IndexedDB) agit comme source de vérité temporaire. Les données sont persistées localement immédiatement.
8. **Détection & File d'attente** : Le Service Worker avec le `Workbox BackgroundSyncPlugin` intercepte les requêtes POST/PUT et les met en file d'attente.
9. **Rejeu & API REST** : Synchronisation vers NestJS dès le retour du réseau.
10. **Stockage permanent** : Validation en base PostgreSQL via Prisma. La base distante devient la source de vérité définitive.

Comportement PWA : Online vs Offline

Opération	Réseau Actif (Online)	Rupture de Réseau (Offline)
Lecture des données	Requête API NestJS directe.	Cache IndexedDB lu via Dexie.js (StaleWhileRevalidate).
Écriture (Ajout produit)	POST immédiat vers la base de données PostgreSQL.	Écriture locale Dexie + mise en file d'attente Workbox.
Feedback UI	Validation serveur immédiate.	Indicateur "En attente de synchronisation" (Optimistic UI).

Intégration matérielle : Module de prise de photo et d'enregistrement des informations de la marchandise

Implémentation technique

Avantage logistique

Responsable : ADDA Kenneth (Membre 6) Le Concepteur matériel et API est chargé du développement de ce module ainsi que de la configuration de l'interconnexion avec les API tierces de taux de change. Ce livrable est prévu au Sprint 2.

Infrastructure et conteneurisation

L'ensemble de l'infrastructure est orchestré via **Docker**, garantissant l'isolation et la reproductibilité entre les environnements de développement et de production. L'architecture se décompose en trois conteneurs indépendants, orchestrés via un fichier `docker-compose.yml` unique.

Structure des répertoires

```
/
|-- docker-compose.yml
|-- /frontend      (NextJs)
|-- /backend       (NestJS)
|-- /database      (PostgreSQL)
```

Rôle de chaque conteneur

Conteneur	Technologie	Rôle
Front-end	Nginx	Sert les assets statiques NextJs (PWA compilée).
Back-end	Node.js / NestJS	Exécute l'application NestJS, traite la logique métier et les calculs Landed Cost.
Base de données	PostgreSQL	Volume persistant isolant les migrations Prisma entre les environnements Dev et Prod.

Organisation de l'équipe (7 rôles distincts)

Pour maximiser l'efficacité des 7 membres du groupe du 6 Mai au 06 Juillet, les responsabilités sont sectorisées de la manière suivante :

M1 HOUEDJISSI David Josias - Product Owner & Scrum Master

Garant de l'alignement des fonctionnalités avec le besoin métier (scénarios "Pour moi" vs "Pour clients"). Pilote l'avancement des sprints, lève les blocages et rédige les cahiers de recette fonctionnelle.

M2 ALLE Marcel - Développeur Backend leader

Architecture de l'API avec NestJS. Implémente la logique de calcul financier complexe, la sécurisation des routes et les algorithmes de répartition des frais annexes.

M3 ADIBA Déo-Gracias - Administrateur de base de données (DBA) & DevOps

Garant de la base PostgreSQL. Gère l'intégrité transactionnelle, écrit les migrations, configure l'environnement conteneurisé avec Docker et orchestre le déploiement final sur serveur.

M4 AGLASSA Denal - Développeur Frontend web (Desktop)

Conçoit et développe l'application web sous NextJs dédiée à la consultation de l'administration, des historiques et des tableaux de bord comptables avancés.

M5 SOSSOU Théodore - Développeur mobile / PWA (Responsive)

Spécialisé dans l'adaptation de l'interface utilisateur NextJs pour l'usage sur smartphone, en veillant à l'ergonomie de l'affichage sur le terrain.

M6 ADDA Kenneth - Concepteur matériel et API

Conçoit le module de prise de photo et d'enregistrement des informations des marchandises et configure l'interconnexion avec les API tierces de taux de change.

M7 GOHOUNGO Nasrallah - Responsable synchronisation & QA (Qualité)

Conçoit les scripts de stockage local pour le mode hors-ligne et rédige la logique de synchronisation automatique lors du retour internet. Supervise les campagnes de tests unitaires.

Feuille de route d'exécution - Phase fondation

Sprint 1 · Initialisation, infrastructure & authentification 6 Mai - 19 Mai 2026

Objectif : Valider l'environnement technique global et sécuriser l'accès à la plateforme.

LIVRABLES

- Dépôts GitHub initialisés et environnements de développement Docker opérationnels.
- **schema.prisma** déployé : schéma PostgreSQL initial avec clés primaires et étrangères.
- Système d'authentification fonctionnel : inscription, connexion par JWT et persistance de session.
- Architecture réseau et API CRUD de base fonctionnelles.

Sprint 2 · Gestion des produits & module matériel 20 Mai - 2 Juin 2026

Objectif : Permettre la saisie des marchandises et implémenter la brique de mobilité.

LIVRABLES

- API Backend et interfaces Frontend pour le CRUD des articles et des catégories.
- Module de prise de photo et d'enregistrement des informations des marchandises (ADDA Kenneth).
- **Stockage local fonctionnel** (IndexedDB via Dexie.js) : mode déconnecté opérationnel.
- Intégration du bouclier Passport/JWT et logique mathématique du calcul métier (Landed Cost).
- Résultat attendu : Authentification sécurisée et 1er flux de colis (Online).

Feuille de route d'exécution - Phase résilience

Sprint 3 · Colisage, devises & moteur de calcul 3 Juin - 16 Juin 2026

Objectif : Gérer l'aspect logistique et automatiser le calcul financier de répartition.

LIVRABLES

- Configuration de Dexie.js avec `useLiveQuery` pour la réactivité immédiate.
- Injection du `Workbox BackgroundSyncPlugin` : application utilisable et synchronisable sans internet.
- Interface de gestion des colis avec assignation dynamique et multiple des articles.
- Interconnexion avec l'API financière pour la conversion automatique des devises vers le Franc CFA (gel du taux en BDD).
- Script de calcul automatique du "Vrai Prix" incluant la ventilation des frais de la table Expenses.

Sprint 4 · Synchronisation, recette & déploiement 17 Juin - 06 Juillet 2026

Objectif : Consolider l'ensemble du projet, chasser les anomalies et livrer la solution sur serveur.

LIVRABLES

- Déploiement du module de prise de photo et application stricte du Design System.
- Algorithme de synchronisation en arrière-plan actif avec gestion et résolution des conflits réseau.
- Tableaux de bord visuels consolidés : historiques des dépenses, budgets restants par voyage.
- Tests de bout en bout, corrections des derniers bugs.
- Déploiement final de l'infrastructure sur serveur VPS - Livraison PWA Production.

Deadline de fin de projet : 06 Juillet 2026 au soir